

將 ASP.NET Core 應用程式部署至 App Engine

來源：<https://codelabs.developers.google.com/codelabs/cloud-app-engine-aspnetcore?hl=zh-tw>

步驟數：10

在本程式碼研究室中，您將瞭解如何將簡單的 ASP.NET Core 應用程式部署至 Google App Engine

目錄

1. [總覽](#)
2. [設定和需求](#)
3. [在 Cloud Shell 中建立 ASP.NET Core 應用程式](#)
4. [執行 ASP.NET Core 應用程式](#)
5. [發布 ASP.NET Core 應用程式](#)
6. [為 App Engine 彈性環境建立 app.yaml](#)
7. [部署至 App Engine 彈性環境](#)
8. [部署新版服務](#)
9. [資訊主頁和流量拆分](#)
10. [恭喜！](#)

1. 總覽

ASP.NET Core 是全新的開放原始碼跨平台架構，可使用 C# 程式設計語言建構新式雲端和網際網路連線應用程式。

在本實驗室中，您將簡單的 ASP.NET Core 應用程式部署至 App Engine 彈性環境。本程式碼研究室以「[從 Google Cloud Shell 建構及啟動 ASP.NET Core 應用程式](#)」程式碼研究室為基礎。建議先完成該實驗室，再進行這個實驗室。

隨著流量和資料儲存需求變化，您可以輕鬆建立、維護及擴充 Google App Engine 應用程式。使用 App Engine 就不需要管理伺服器。只要上傳應用程式，就能開始使用。

App Engine 應用程式會根據傳入的流量自動調度資源。App Engine 原生支援負載平衡、微服務、授權、SQL 和 NoSQL 資料庫、Memcache、流量拆分、記錄、搜尋、版本管理、推出和回溯，以及安全掃描，而且所有功能都可高度自訂。

App Engine 的環境 ([標準環境](#)和[彈性環境](#)) 支援多種程式設計語言，包括 C#、Java、Python、PHP、Node.js、Go 等。這兩種環境各有優點，可讓使用者彈性選擇應用程式的運作方式。詳情請參閱「[選擇 App Engine 環境](#)」。

課程內容

- 如何將簡單的 ASP.NET Core 應用程式封裝為 Docker 容器。
- 如何將簡單的 ASP.NET Core 應用程式部署至 App Engine。

軟硬體需求

- Google Cloud Platform 專案
 - [Chrome](#) 或 [Firefox](#) 瀏覽器
-

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' button circled in blue. Below this, the 'Select a project' section includes a search bar. The 'New Project' section is below that, featuring a 'Project name' field with 'my-kubernetes-codelab' entered, which is also circled in blue. Below the name field is the 'Project ID' field, which contains 'my-kubernetes-codelab' and a note that it cannot be changed later. Below that is the 'Location' field with 'No organization' selected. At the bottom are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串，您隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中不得重複，且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常會標示為 `PROJECT_ID`)，因此如果您不喜歡該字串，可以產生另一個隨機字串，或是嘗試使用自己的字串，看看是否可用。專案建立後，系統就會「凍結」該值。
- 還有第三個值，也就是部分 API 使用的「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中不得重複，選定後即為專屬 ID，其他使用者無法使用。您是該 ID 的唯一使用者。專案刪除後，該 ID 就無法再使用。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用應該不高，甚至完全免費。如要停用資源，避免在本教學課程結束後繼續產生帳單費用，請按照程式碼研究室結尾的「清除」操作說明

操作。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

啟用 Cloud Shell

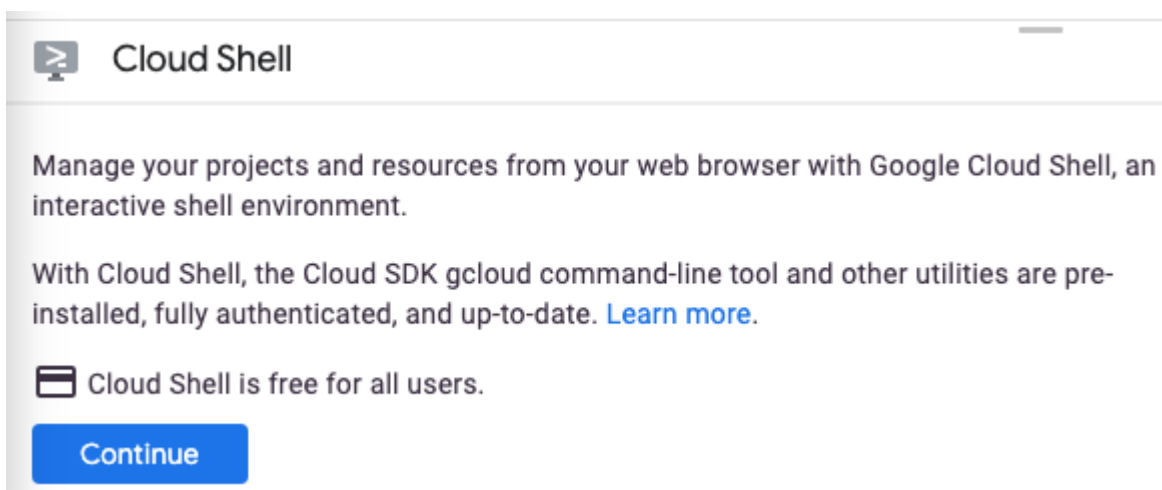
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



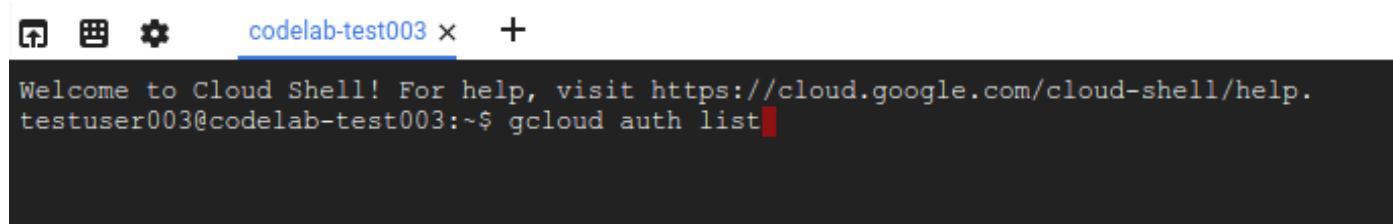
。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面 (位於摺疊式選單下方)，說明這個指令列環境。點選「繼續」後，這則訊息日後就不會再出現。以下是這個初次畫面的樣子：



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器搭載您需要的所有開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器或 Chromebook 上完成。

連線至 Cloud Shell 後，您應會發現自己通過驗證，且專案已設為您的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

Credentialed Accounts

ACTIVE ACCOUNT

* <my_account>@<my_domain.com>

To set the active account, run:

```
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]  
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

步驟 3 · 約 2 分鐘

3. 在 Cloud Shell 中建立 ASP.NET Core 應用程式

在 Cloud Shell 提示中，列出已安裝的 .NET SDK，即可確認 `dotnet` 指令列工具是否已安裝：

```
dotnet --list-sdks
```

接著，建立新的 ASP.NET Core 網頁應用程式基本架構，目標架構為 `netcoreapp3.1`：

```
dotnet new mvc -o HelloWorldAspNetCore -f netcoreapp3.1
```

這個指令會建立專案並還原依附元件，顯示的訊息應如下所示。

```
Restore completed in 11.44 sec for HelloWorldAspNetCore.csproj.
```

```
Restore succeeded.
```

步驟 4 · 約 3 分鐘

4. 執行 ASP.NET Core 應用程式

我們即將執行應用程式。請前往應用程式資料夾。

```
cd HelloWorldAspNetCore
```

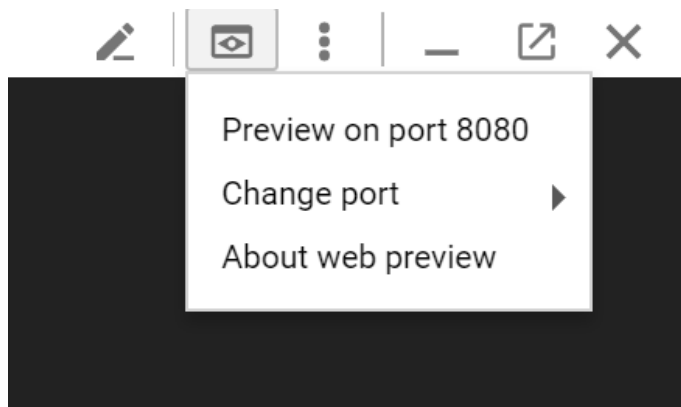
最後，執行應用程式。

```
dotnet run --urls=http://localhost:8080
```

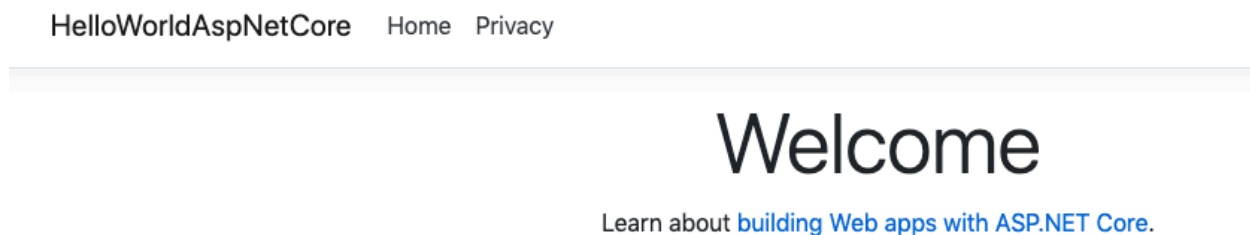
應用程式會開始監聽通訊埠 8080。

```
Hosting environment: Production
Content root path: /home/atameldev/HelloWorldAspNetCore
Now listening on: http://[::]:8080
Application started. Press Ctrl+C to shut down.
```

如要確認應用程式是否正在執行，請點按右上方的網頁預覽按鈕，然後選取「透過以下通訊埠預覽：8080」。



您會看到預設的 ASP.NET Core 網頁：



確認應用程式正在執行後，請按下 Ctrl+C 鍵關閉應用程式。

步驟 5 · 約 2 分鐘

5. 發布 ASP.NET Core 應用程式

現在，請使用 `dotnet publish` 指令發布應用程式，取得獨立的 DLL。

```
dotnet publish -c Release
```

執行 `publish` 會顯示一些訊息，程序結束時，會顯示已成功發布的 DLL。

```
...
HelloWorldAspNetCore ->
/home/ataameldev/HelloWorldAspNetCore/bin/Release/netcoreapp3.1/HelloWorldAspNetCore.dll
HelloWorldAspNetCore ->
/home/ataameldev/HelloWorldAspNetCore/bin/Release/netcoreapp3.1/HelloWorldAspNetCore.Views.dll
HelloWorldAspNetCore ->
/home/ataameldev/HelloWorldAspNetCore/bin/Release/netcoreapp3.1/publish/
```

步驟 6 · 約 2 分鐘

6. 為 App Engine 彈性環境建立 app.yaml

`app.yaml` 檔案說明如何將應用程式部署至 App Engine，在本例中為 App Engine 彈性環境。

首先，前往 `publish` 資料夾。這應該位於 `bin/Release` 資料夾下方，但確切路徑取決於 .NET 版本：

```
cd bin/Release/netcoreapp3.1/publish/
```

在 `publish` 資料夾中建立 `app.yaml` 檔案：

```
cat <<EOT >> app.yaml
env: flex
runtime: aspnetcore
EOT
```

請注意，`app.yaml` 檔案會將環境指定為 `flex`，並將執行階段指定為 `aspnetcore`。

步驟 7 · 約 5 分鐘

7. 部署至 App Engine 彈性環境

您已準備好使用 `gcloud` 將應用程式部署至 App Engine 彈性環境。在 `publish` 目錄中執行下列指令：

```
gcloud app deploy --version v0
```

部署期間，系統可能會要求您選擇應用程式的地區。選擇要讓應用程式在哪個地區執行。

```
Please choose a region for your application. After choosing a region,
you cannot change it. Which region would you like to choose?
```

- [1] europe-west (supports standard and flexible)
- [2] us-central (supports standard and flexible)
- [3] us-east1 (supports standard and flexible)
- [4] asia-northeast1 (supports standard and flexible)
- [5] cancel

注意事項

首次部署可能需要幾分鐘的時間。這是因為 App Engine 彈性環境會在幕後自動為您佈建 Google Compute Engine 虛擬機器，然後安裝並啟動應用程式。如果部署作業逾時，請通知指導老師。

這會在雲端為應用程式建立映像檔、將該映像檔儲存至 Google Container Registry，並部署至 App Engine。部署期間，您會看到容器映像檔的建構過程：

```
Operation completed over 1 objects/571.8 KiB.
BUILD
Step #0: Pulling image: gcr.io/gcp-
runtimes/aspnetcorebuild@sha256:d7b7975acb374fc3a9655a4e529993e6270cfa78023885684626528bc379f
8eb
Step #0: sha256:d7b7975acb374fc3a9655a4e529993e6270cfa78023885684626528bc379f8eb: Pulling
from gcp-runtimes/aspnetcorebuild
```

最後，您應該會看到應用程式已部署。

```
...
Deployed service [default] to [https://<project-id>.appspot.com]
```

部署應用程式後，請在網路瀏覽器中開啟網址 `http://<project-id>.appspot.com`，即可造訪應用程式。

注意事項

如果不記得 `project-id`，請參閱「設定和需求」一節，或檢查部署記錄中是否有類似下列內容的項目：

```
Deployed service [default] to [https://<project-id>.appspot.com]
```

注意事項

您也可以選擇為應用程式購買及使用頂層網域名稱，或使用您已註冊的網域名稱。

新分頁會顯示預設的 ASP.NET Core 網頁。

HelloWorldAspNetCore Home Privacy

Welcome

Learn about building Web apps with ASP.NET Core.

您也可以查看在雲端為您建立的容器映像檔。在 Cloud 控制台中，依序前往「Container Registry」>「Images」，然後在 appengine 資料夾中，您應該會看到應用程式的映像檔。

← Images

REFRESH

SHOW PULL COMMAND

DELETE

eu.gcr.io / ddc-dotnet17-cgn-2008 / appengine / default.v0

Filter by name or tag

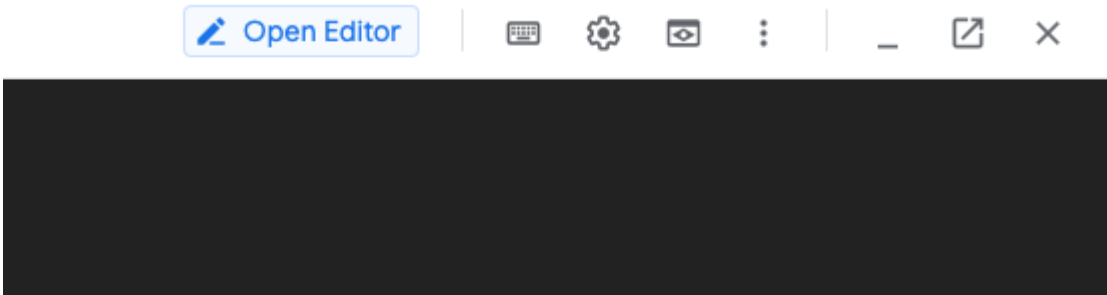
Columns

☐	Name	Tags	Virtual size	Uploaded	Build	Vulnerabilities	Type
☐	 c8ea21dc65d0	latest 	208 MB	3 minutes ago	c75d5e30-814e...	 Not scanned	Docker Manifest, Schema 2

8. 部署新版服務

您部署到實際工作環境的應用程式，在某個時間點會需要修正錯誤或新增功能。App Engine 可協助您將新版本部署至正式環境，同時不影響使用者。

首先，請修改應用程式。從 Cloud Shell 開啟程式碼編輯器。



前往 `HelloWorldAspNetCore` 的 `Views/Home` 資料夾下的 `Index.cshtml`，然後將預設訊息更新為：

```
Learn about <a href="https://docs.microsoft.com/aspnet/core">building Web apps with ASP.NET Core on Google Cloud Platform!
```

儲存變更，然後返回 Cloud Shell。在 `HelloWorldAspNetCore`，中發布應用程式，取得獨立的 DLL。

```
dotnet publish -c Release
```

前往發布目錄。

```
cd bin/Release/netcoreapp3.1/publish/
```

您現在可以部署新版應用程式 (在本例中為 `v1`)。

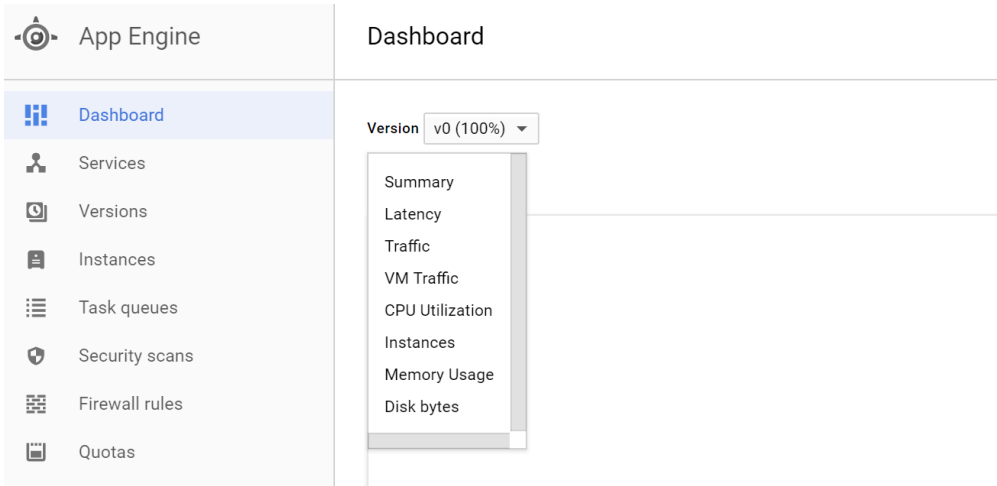
```
gcloud app deploy --version v1
```

部署完成後，您可以前往 Google Cloud 控制台的 App Engine 版本專區，查看應用程式的新版本，該版本會提供所有流量和新訊息。

Filter versions							
☐ Version	Status	Traffic Allocation	Instances	Runtime	Environment		
☐ v1	Serving	100%	2	aspnetcore	Flexible		
☐ v0	Stopped	0%	2	aspnetcore	Flexible		

9. 資訊主頁和流量拆分

在 App Engine 的「資訊主頁」部分下方，您可以看到應用程式的延遲時間、CPU 等多個資訊主頁。請自行探索這些資訊主頁。



在「版本」部分下方，您會看到已部署的應用程式版本，並可在「流量拆分」部分拆分不同版本之間的流量。現在要拆分兩個版本之間的流量：

← Split traffic

You can split incoming traffic to different versions of your app. Traffic splitting is useful for slowly rolling out new versions or A/B testing different designs and features [Learn more](#)

Split traffic by

☒ IP address

?

☐ Cookie

?

☐ Random

?

Traffic allocation

v0

will receive the remaining

50 %

×

v1

50 %

×

+ Add version

Save

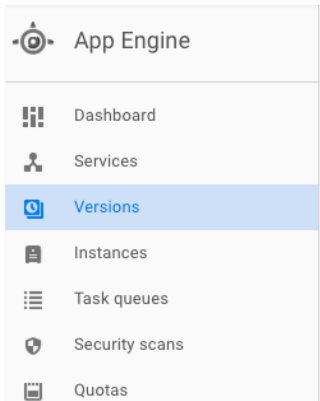
Cancel

10. 恭喜！

Cleanup

現在該關閉應用程式，節省成本並成為優質的雲端使用者。

前往 App Engine 的版本部分。



選取版本並停止。

Versions					REFRESH	DELETE	STOP	START
More Columns ▾								
☒	Version	Status	Traffic Allocation	Instances				
☒	20161206t153948 	Serving	100%	2				

停止版本後，系統會刪除支援執行個體，執行個體數量應會降至零。

Versions					REFRESH	DELETE	STOP	START
More Columns ▾								
☒	Version	Status	Traffic Allocation	Instances				
☒	20161206t153948 	Stopped	100%	0				

涵蓋內容

好了！您已建立 ASP.NET Core 應用程式，並封裝為 Docker 容器，然後部署到 Google App Engine 彈性環境。

- 如何將簡單的 ASP.NET Core 應用程式封裝為 Docker 容器。
- 如何將簡單的 ASP.NET Core 應用程式部署至 App Engine。

後續步驟

- 進一步瞭解 [Google Cloud Platform 上的 Windows](#)。
- 進一步瞭解 [Google Cloud Platform 上的 .NET](#)。
- 進一步瞭解 [Google Cloud Platform 上的 SQL Server](#)。
- 進一步瞭解 [Cloud Tools for Visual Studio](#)。
- 進一步瞭解 [Cloud Tools for PowerShell](#)。

授權

這項內容採用的授權為 [Creative Commons 姓名標示 2.0 通用授權](#)。
